

Understanding the STM32F767ZI Servo PWM Demo

What you actually built, why every number is what it is, and what it proves

Companion guide for the 6-DOF robotic arm project · Microcontroller Interfacing, Semester 1/2026

Board: NUCLEO-F767ZI · Timer: TIM3 Channel 1 · Pin: PC6

0. How to read this guide

You have working code and a debugger showing the right numbers. This document explains why those numbers are correct rather than lucky, starting from the assumption that you have not used an STM32 before.

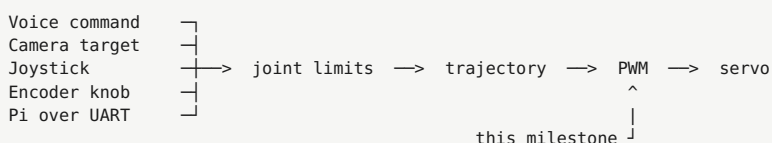
Read section 1 for where this fits in the arm project, section 2 as a dictionary you can flip back to, and sections 3 through 7 in order. Sections 3 to 7 build one continuous chain of reasoning from a crystal oscillator to a pulse on a pin. Nothing in that chain is skipped.

1. Where this milestone sits in the whole project
2. Vocabulary: every abbreviation, decoded
3. What a hobby servo actually wants
4. Where the clock comes from
5. How a timer turns a clock into a pulse
6. Deriving $PSC = 107$ and $ARR = 19999$
7. From an angle to a compare value
8. What your code does, layer by layer
9. Reading the debugger output
10. What is proven, and what is not
11. Calibration versus feedback
12. Extending to six servos
13. Questions you should be able to answer

1. Where this milestone sits in the whole project

The full system is a six-axis robotic arm with a Raspberry Pi doing vision and speech, an STM32 doing real-time control, a joystick, an LCD, and an emergency stop. That is a large amount of machinery, and almost none of it works without one thing being true first: **the STM32 must be able to command a servo position reliably.**

Every operating mode in your architecture ends at the same place. Auto/Voice mode, Cartesian mode, Joint Group mode and Single-Joint Service mode all produce a target joint angle, and all of them feed that angle into the same motion pipeline, which ends in a PWM signal. If that final stage is unreliable, every mode above it is unreliable too, and no amount of good inverse kinematics will save it.



So this is not a warm-up exercise. It is the foundation the other fifteen items on your development priority list stand on. It is also the part your course explicitly grades: the requirement list names PWM as one of the three required modules, alongside ADC and the graphic LCD.

What you completed

Step 2 of your development priority list: *test one servo with STM32 hardware PWM*. You did it without a servo attached, which is the correct order. Software correctness first, hardware second. A bug found on a scope trace costs minutes; the same bug found through a stripped servo gear costs a part.

2. Vocabulary: every abbreviation, decoded

STM32 documentation is dense with three-letter names. Here is every one that appears in your project so far. You do not need to memorise this; you need to be able to look it up until it sticks.

2.1 The chip and the tools

Term	Stands for	What it means for you
MCU	Microcontroller Unit	The whole chip: CPU, memory and peripherals on one piece of silicon. Yours is the STM32F767ZI.
HAL	Hardware Abstraction Layer	ST's C library that hides register bit-fiddling behind functions like <code>HAL_TIM_PWM_Start()</code> .
CubeMX	—	The graphical configurator. You click pins and clocks; it writes the setup code.
CubeIDE	—	The editor, compiler and debugger, with CubeMX built into it.
ST-Link	—	The debugger chip built onto the top half of your Nucleo board. It flashes and debugs the target chip over USB.
GPIO	General Purpose Input/Output	A physical pin the software can control. PC6 means port C, pin 6.
AF	Alternate Function	A mode where a pin is handed over to a peripheral instead of plain software control. <code>GPIO_AF2_TIM3</code> means "PC6 now belongs to TIM3".
DOF	Degrees of Freedom	Independent axes of motion. Six DOF means six joints, so six servos.

2.2 Clock words

Term	Stands for	What it means for you
HSI	High Speed Internal	An oscillator built into the chip, 16 MHz, accurate to roughly $\pm 1\%$. Your project currently uses this.
HSE	High Speed External	A clock from outside the chip. On a Nucleo it comes from the ST-Link at 8 MHz and is far more accurate.
PLL	Phase-Locked Loop	A frequency multiplier. It takes a slow input and produces a fast, locked output. This is how 16 MHz becomes 216 MHz.
SYSCLK	System Clock	The master clock of the whole chip after the PLL. Yours is 216 MHz, the maximum for this part.
HCLK	AHB Clock	The clock feeding the CPU core and the main internal bus. Yours is 216 MHz.
AHB	Advanced High-performance Bus	The fast internal highway connecting CPU, memory and DMA.
APB	Advanced Peripheral Bus	Slower side-roads off the AHB where peripherals live. There are two: APB1 and APB2.
PCLK1	Peripheral Clock 1	The APB1 bus clock. Yours is 54 MHz.
Prescaler	—	A divider. "APB1 prescaler /4" means APB1 runs at one quarter of HCLK.

2.3 Timer words (the important ones)

Term	Stands for	What it means for you
TIM	Timer	A counting peripheral. TIM3 is timer number 3. The F767 has fourteen of them.
CNT	Counter	The register holding the timer's current count. It increments on every timer clock tick.
PSC	Prescaler	Divides the incoming clock before it reaches the counter. Controls <i>how fast</i> CNT increments.
ARR	Auto-Reload Register	The value CNT resets at. Controls <i>how long</i> one cycle lasts.
CCR	Capture/Compare Register	The value CNT is compared against. Controls <i>the pulse width</i> . CCR1 belongs to channel 1.
PWM	Pulse Width Modulation	A repeating on/off signal where the meaning is carried by how long the "on" part lasts.
OC	Output Compare	The timer feature that drives a pin based on comparing CNT against CCR.
Update event	—	The moment CNT wraps from ARR back to 0. Buffered register changes take effect here.
Preload	—	Buffering. A new ARR or CCR value waits in a shadow register and is applied at the update event, never mid-pulse.

2.4 Words from the wider project

Term	Stands for	What it means for you
ADC	Analog to Digital Converter	Turns a voltage into a number. Your joystick will use two channels of it.
DMA	Direct Memory Access	Moves data between a peripheral and memory without the CPU doing the copying.
EXTI	External Interrupt	A pin change that interrupts the CPU immediately. Your buttons and e-stop will use this.
NVIC	Nested Vectored Interrupt Controller	The hardware that decides which interrupt runs and in what priority order.
UART	Universal Asynchronous Receiver/Transmitter	The serial link that will carry commands from the Raspberry Pi.
SPI	Serial Peripheral Interface	A fast synchronous bus. Your graphic LCD will use it.
I2C	Inter-Integrated Circuit	A two-wire bus. Relevant only if you later add magnetic encoders.
MPU	Memory Protection Unit	Cortex-M7 feature restricting memory access. CubeMX configured it for you; it does not affect your timer.

3. What a hobby servo actually wants

Before any STM32 detail, understand what the target device expects, because every number downstream is derived from it.

A hobby servo (MG996R, SG90 and similar) has three wires: power, ground, and signal. The signal wire is **not** a power connection and carries almost no current. It carries a repeating pulse, and the servo measures how long each pulse stays high.

Pulse high time	Conventional meaning
1.0 ms	one end of travel, commonly called 0°
1.5 ms	centre, commonly called 90°
2.0 ms	the other end, commonly called 180°

The pulse repeats roughly every 20 ms, which is 50 times per second. That repetition rate is a refresh: the servo needs to keep being told where to go, and if the pulses stop, most servos go limp or hold weakly.

The three facts that matter

1. **Only the high time carries information.** Not the voltage, not the duty cycle percentage, not the average. A 1.5 ms pulse means centre whether the period is 20 ms or 18 ms.
2. **The timing must be accurate to microseconds.** The whole useful range is only 1000 microseconds wide, spread across 180 degrees. That is about 5.6 microseconds per degree. Sloppy timing is visible as jitter.
3. **The signal is a request, not a command.** The servo has its own internal controller that decides how to reach the requested position. You are talking to that controller, not to the motor.

Point 2 is the reason your architecture forbids generating this signal with software delays. If the CPU is busy drawing the LCD when a pulse should end, the pulse gets longer, and the servo twitches. A hardware timer does not have that problem, because it is not running your code.

4. Where the clock comes from

Everything in an STM32 is driven by a clock. To know how long a timer tick lasts, you have to trace the clock backwards to its source. Here is your actual chain, taken from the code CubeMX generated in `SystemClock_Config()`.

```
HSI oscillator                16 MHz
|
├─ ÷ PLLM (8)                2 MHz    PLL input
├─ × PLLN (216)              432 MHz   VCO output
└─ ÷ PLLP (2)                216 MHz   SYSCLK
    |
    ├─ ÷ AHB prescaler (1)    216 MHz   HCLK → CPU, memory
    |
    ├─ ÷ APB1 prescaler (4)   54 MHz    PCLK1 → peripherals on APB1
    |
    └─ × 2 (timer rule)      108 MHz   APB1 timer clock → TIM3
```

4.1 Why there is a ×2 at the end

This is the step that confuses everyone the first time. Peripherals on APB1 run at 54 MHz, but the *timers* on APB1 run at 108 MHz. The rule built into every STM32 is:

If the APB prescaler is 1, the timer clock equals the APB clock.
If the APB prescaler is anything other than 1, the timer clock is **twice** the APB clock.

Your APB1 prescaler is 4, which is not 1, so the doubling applies. ST designed it this way so timers keep their resolution even when the peripheral bus is slowed down to meet its maximum frequency limit. APB1 on this chip cannot exceed 54 MHz, which is why the prescaler is 4 in the first place.

You can see this labelled directly in the CubeMX Clock Configuration screen: there is a box reading *APB1 peripheral clocks* showing 54, and next to it a separate box reading *APB1 timer clocks* showing 108. Those are two different numbers for a reason.

4.2 Why TIM3 and not some other number

TIM3 sits on APB1. TIM1 and TIM8 sit on APB2, which in your configuration runs its timers at 216 MHz. Which bus a timer lives on changes its input clock, so it changes the prescaler you need. This is why the correct workflow was to look at your clock tree first rather than copying a prescaler value from a tutorial written for a different chip.

A note on HSI versus HSE

Your PLL currently runs from HSI, the internal RC oscillator. It is convenient and needs no external part, but its accuracy is around $\pm 1\%$ and it drifts with temperature. At $\pm 1\%$, a pulse you intend to be 1500 μs could actually be anywhere from 1485 to 1515 μs , which is roughly $\pm 3^\circ$ of commanded position. Switching the PLL source to HSE (8 MHz from the ST-Link, with PLLM = 4) removes that error, because it comes from a crystal. Worth doing before you spend time calibrating, so you are not calibrating against a moving target.

5. How a timer turns a clock into a pulse

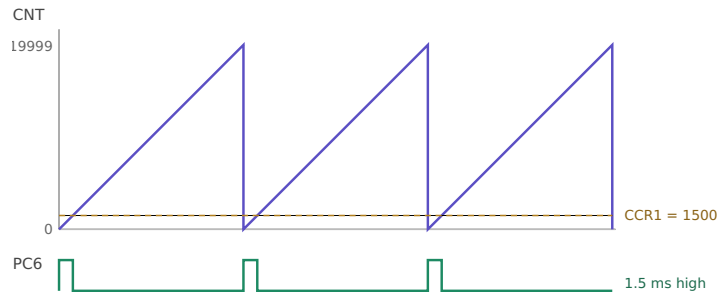
A timer is, at heart, a counter that counts clock ticks. Everything else is arranged around that counter. Four registers

matter.

5.1 The counter and its two boundaries

`CNT` is the counter. It starts at 0 and adds 1 on every tick of the timer's clock. When it reaches `ARR`, it wraps back to 0 and starts again. That wrap is called the update event. So `ARR` sets the length of one cycle.

`PSC` sits in front of the counter and divides the incoming clock. If `PSC` is 0, the counter gets every tick. If `PSC` is 107, the counter gets one tick for every 108 that arrive. The divisor is `PSC + 1`, not `PSC`, because a prescaler of 0 must mean "divide by 1" rather than "divide by nothing".



The counter ramps to `ARR` and resets. The pin is high only while `CNT` is below `CCR1`.

5.2 The comparison that drives the pin

Each timer channel has its own `CCR`. In PWM mode 1, the hardware continuously compares `CNT` against `CCR` and drives the output pin high while `CNT < CCR`, low otherwise. That is the entire mechanism.

Notice what this gives you. The period is set once by `ARR` and shared. The pulse width is set per-channel by `CCR` and can be changed at any time. That maps perfectly onto servos, which all want the same 20 ms period but individual pulse widths.

5.3 Why this runs without the CPU

The counter, the comparator and the pin driver are all silicon. Once `HAL_TIM_PWM_Start()` has enabled them, they operate whether your program is running, waiting, or halted at a breakpoint. Your software's only job is to occasionally write a new number into `CCR`.

You can demonstrate this to yourself: while debugging, press Suspend. The CPU stops, your sweep stops updating, but TIM3 keeps counting and PC6 keeps emitting the last pulse width you commanded. That is the concrete difference between hardware PWM and toggling a pin in software.

6. Deriving PSC = 107 and ARR = 19999

Now the two numbers can be derived rather than guessed. The strategy is to choose a counter speed that makes the arithmetic human-readable, then choose a period.

6.1 Step one: make one tick equal one microsecond

Servo datasheets talk in milliseconds and microseconds. If the counter also counts in microseconds, then `CCR` values and pulse widths are the same number and no conversion is needed anywhere in the code. So the target is a counter frequency of 1 MHz, which is one tick per microsecond.

```
required divisor = timer clock / target counter frequency
                  = 108,000,000 / 1,000,000
                  = 108

PSC = divisor - 1 = 107
```

The subtraction is the part people get wrong. The hardware divides by `PSC + 1`. Writing 108 would give a divisor of 109 and a counter at 990,826 Hz, which is close enough to look correct on a scope and wrong enough to matter after calibration.

6.2 Step two: make one cycle last 20 milliseconds

```
20 ms = 20,000 μs = 20,000 ticks  
  
counter counts 0, 1, 2, ... , ARR → that is ARR + 1 distinct values  
  
ARR + 1 = 20,000  
ARR      = 19999
```

The off-by-one has the same cause as before: counting includes zero. This gives a PWM frequency of $1,000,000 / 20,000 = 50$ Hz exactly, which is what the servo expects.

6.3 Step three: the pulse width

With a 1 μs tick, CCR is the pulse width in microseconds directly. No formula, no scaling.

CCR1 value	Pulse width	Duty cycle	Command
1000	1.0 ms	5.0 %	0°
1500	1.5 ms	7.5 %	90°
2000	2.0 ms	10.0 %	180°

Resolution, and why 1 MHz was a good choice

The useful range spans 1000 ticks across 180°, giving 0.18° per tick. That is finer than any hobby servo can mechanically resolve, so the timer is not your limiting factor. Had you chosen a 10 MHz counter you would have gained resolution you cannot use, and ARR would have needed 199,999 — which does not fit in TIM3's 16-bit registers (maximum 65,535). A 100 kHz counter would have fit, but each tick would be 10 μs, and your whole 180° range would collapse to 100 steps of 1.8°. 1 MHz is the sweet spot for a 16-bit timer doing servo work.

7. From an angle to a compare value

The hardware speaks in microseconds. Your application wants to speak in degrees. One function bridges them, and it is deliberately the only place that conversion happens.

```
pulse_us = min_pulse_us + (angle / 180) * (max_pulse_us - min_pulse_us)
```

With min = 1000 and max = 2000, an angle of 90 gives $1000 + 0.5 \times 1000 = 1500$. That value goes straight into CCR1.

The important design decision is that `min_pulse_us` and `max_pulse_us` live in a per-servo table rather than being fixed constants. Two MG996R servos from the same bag will not have identical end stops. One might reach its mechanical limit at 1040 μs and the other at 985 μs. When you eventually measure that, you change two numbers in a table and every angle command for that joint becomes more accurate, with no change to the conversion code.

8. What your code does, layer by layer

There are four layers between your application and the pin. Keeping them separate is what allows this demo to grow into a six-joint arm without being rewritten.

Layer	File	Responsibility
Demo / trajectory	<code>servo_demo.c</code>	Decides what angle to command and when. Interpolates smoothly between waypoints.
Servo driver	<code>servo_driver.c</code>	Converts angle to pulse width using per-servo calibration; clamps; writes CCR.
HAL and timer	<code>tim.c</code>	Configures TIM3 with PSC, ARR and PWM mode; enables the channel.
Hardware	—	Counts, compares, drives PC6.

8.1 The runtime verification in `Servo_Init()`

Rather than trusting that CubeMX kept your settings, the driver checks them when it starts:

```

timer_hz      = actual clock, read from RCC->CFGR at runtime
counter_hz    = timer_hz / (htim3.Init.Prescaler + 1)

if (counter_hz != 1,000,000)          → SERVO_ERR_TIMER_CLOCK
if (htim3.Init.Period != 19,999)      → SERVO_ERR_PERIOD

```

This matters because CubeMX regenerates code every time you change a pin, and a reset field is easy to miss. A wrong prescaler produces a signal that looks plausible but is silently the wrong speed. This check turns a subtle timing bug into an immediate, obvious failure at startup.

8.2 Why the sweep does not use HAL_Delay()

`HAL_Delay(1000)` blocks. For one second the CPU does nothing else: no UART, no LCD, no button check, no emergency stop response. In a robot arm that is unacceptable, and your architecture document forbids it explicitly.

Instead `ServoDemo_Update()` is called continuously from the main loop and returns immediately unless 20 ms have passed since its last action. It keeps its position in the movement using variables rather than by blocking. That is the same structure the real trajectory generator will use, so this code grows rather than gets thrown away.

```

while (1)
{
    ServoDemo_Update();    // returns in a few instructions if it is not time yet
    // later: UART_Poll(); LCD_Update(); Joystick_Read(); Safety_Check();
}

```

9. Reading the debugger output

Your Live Expressions view showed five values. Here is what each one proves.

Expression	Value	What it proves
TIM3->PSC	107	The prescaler reached the hardware. The counter is running at 1 MHz.
TIM3->ARR	19999	The period is 20 ms, so the output frequency is 50 Hz.
TIM3->CNT	1477	The counter is alive. The value itself is meaningless; you simply caught it 1477 μ s into a ramp. A frozen CNT would mean the timer clock was never enabled.
TIM3->CCR1	1000	Your software successfully commanded a 1.0 ms pulse, which the calibration table maps to 0°.
s_counter_hz	1000000	The driver independently walked the clock tree at runtime and confirmed 1 MHz. This is not a number you typed; it is a number the code derived.

Watching CCR1 move between 1000 and 2000 on a repeating cycle is the complete proof for this milestone. It demonstrates that an angle command travels all the way through the conversion, the clamping and the register write, and lands as a different pulse width on a real pin.

Why CNT looks random

CNT completes a full 0 to 19999 ramp every 20 ms, which is 50 times a second. The debugger polls roughly once per second. Every read catches the counter at an unrelated point in its ramp, so consecutive readings look like noise. That is expected and is itself evidence the timer is free-running rather than stalled.

10. What is proven, and what is not

Being precise about this is what separates an engineering claim from a hopeful one, and it is the kind of distinction that earns marks.

Proven

- The clock tree is understood and the timer's input frequency is known, not assumed.
- The counter runs at exactly 1 MHz, verified by code at runtime.
- The period is exactly 20 ms, giving 50 Hz.
- An angle command is converted to a pulse width and reaches the compare register.
- The signal is generated by hardware and continues independently of the CPU.

- The pin PC6 is correctly switched to alternate function AF2 and owned by TIM3.

Not proven

- **That the physical waveform is correct.** Registers say what the hardware intends. Only an oscilloscope or logic analyser on PC6 confirms what actually comes out.
- **That a servo would move to 90° when sent 1500 µs.** That is a property of the servo, not of your code. The mapping is a convention, not a guarantee.
- **That the arm can hold a position under load.** Nothing here involves torque, power supply behaviour, or mechanical stiffness.

The single most common overclaim in student servo projects is "I sent 90 degrees so the joint is at 90 degrees". Your architecture document already handles this correctly: it says a completion message means the planned movement time has elapsed, not that a position was reached. Keep that distinction in your report.

11. Calibration versus feedback

These two are often confused. They solve different problems and one does not replace the other.

Calibration	Closed-loop feedback
Done once, offline. You measure where a specific servo actually goes for a few pulse widths, then store a corrected mapping.	Runs continuously. You measure actual position, compare it to the target, and adjust the command to shrink the error.
Needs a protractor and patience. No extra electronics.	Needs a sensor on every joint: a potentiometer plus ADC, or a magnetic encoder over I2C.
Fixes systematic error: this servo's centre is at 1530 µs, not 1500.	Fixes dynamic error: the arm sagged under load and needs more command to hold position.
<code>Servo_SetCalibration()</code> already supports this.	Not in your architecture. Section 8 states the arm has no external joint encoders.

Your handoff document discussed adding potentiometers or AS5600 encoders. Be aware this conflicts with the final architecture, and there are practical obstacles: your ADC channels are already assigned to the joystick, and six AS5600 sensors share one fixed I2C address, so they would need a multiplexer. Treat feedback as an optional extension on a single joint, not a planned feature.

How you will calibrate, when hardware arrives

1. Power the servo from its own 6 V supply, never from the Nucleo. Connect grounds together.
2. Use `Servo_SetPulseUs()` to send raw widths, starting near 1500 and moving outward slowly.
3. Find the pulse width where the servo just reaches each mechanical end, then back off slightly. Driving a servo into its end stop makes it buzz and draws stall current.
4. Record those two values and pass them to `Servo_SetCalibration()` for that joint.
5. Repeat per joint. The base and the gripper will not have the same numbers.

12. Extending to six servos

Nothing in the driver is single-channel specific. TIM3 has four channels sharing one counter, so four servos get the same 20 ms period automatically while keeping independent pulse widths. TIM4 provides the remaining two.

Joint	Timer and channel	Suggested pin
J1 base	TIM3_CH1	PC6 (done)
J2 shoulder	TIM3_CH2	PC7
J3 elbow	TIM3_CH3	PC8
J4 wrist pitch	TIM3_CH4	PC9
J5 wrist rotation	TIM4_CH1	PD12
J6 gripper	TIM4_CH2	PD13

Pins to avoid: PA6 and PB4 are alternatives for TIM3_CH1 but collide with SPI, which your LCD needs. PB0 and PB7 are alternatives for other channels but are wired to the Nucleo's onboard LEDs. Reserve TIM2 for the rotary encoder, which needs a timer in encoder mode.

The code change per joint is three edits: enable the channel in CubeMX, uncomment the entry in the `Servo_Id_t` enum, and add a row to the calibration table. TIM4 will need its own PSC of 107 and ARR of 19999, and the runtime check will catch it if you forget.

One hardware consequence worth knowing

All four TIM3 channels begin their pulse at the same instant, when the counter passes zero. Four servos therefore start drawing current simultaneously, so the peak load on the 6 V rail is higher than the average suggests. This is a reason to keep the bulk capacitance on that rail generous, and a reason the architecture puts servos on a separate supply from logic.

13. Questions you should be able to answer

If you can answer these without notes, you understand this milestone. If one of them makes you hesitate, that section is worth rereading.

1. Where does the 108 MHz feeding TIM3 come from, and why is it not 54 MHz?
2. Why is PSC 107 rather than 108?
3. Why is ARR 19999 rather than 20000?
4. What would happen to the output if you doubled ARR and left PSC alone?
5. Why does a servo care about the high time and not the duty cycle percentage?
6. What does $CCR1 = 1750$ correspond to, in microseconds and in degrees?
7. Why can four servos share TIM3 but still hold four different positions?
8. Why does the PWM signal continue when the debugger halts the CPU?
9. What is the difference between calibrating a servo and closing a feedback loop?
10. What have you actually proven about the servo's physical position? (Answer: nothing.)

Where you are

Development priority step 2 is complete. Step 3 is all six PWM outputs with no arm attached, which is mostly CubeMX work plus table entries. Before that, two small things are worth doing: switch the PLL source to HSE for timing accuracy, and put a logic analyser or oscilloscope on PC6 to confirm the physical waveform matches what the registers claim.